

감사렌즈 기술 산출물 문서

코스피 전 종목 감사보고서 RAG 서비스 — 기술 명세 · 설계 근거 · 경험 기록
스터디·면접 대비용 | 설계서 v0.1(2026-06-24) → 라이브 서비스(2026-07-21)의 전 과정

이 문서는 감사렌즈의 구현된 그대로를 기록한 기술 산출물이다. 각 기술마다 ①무엇을 ②왜 선택했고 ③실제로 무엇을 겪었는지(경험)를 함께 적어, 면접에서 '써봤다'가 아니라 '겪어봤다'로 말할 수 있게 구성했다. 최초 설계서(audit_rag_design.md)와 개발 종합문서(62쪽)를 원천으로 하며, 마지막 장의 '설계 대비 변경 이력'과 '면접 Q&A; 10선'이 이 문서의 백미다. 작성일: 2026-07-21.

1. 서비스 최종 스펙 — 한 장 요약

항목	최종 구현
서비스	코스피 808사 감사보고서·재무제표 주석 RAG — 자연어 질의 → 근거(원문 인용+DART 딥링크) 부착 답변
데이터	3개 사업연도 연차(감사) + 최근 분·반기(검토), 3,208개 보고서 → 604,458 청크(v2) + Fact 13,523건·18종
검색	6경로 라우팅: Fact Store SQL(전수) / 하이브리드 벡터 / 온디맨드 추출 / 지식그래프 / 정형수치(XBRL) / 유사사례
LLM	역할별 4계층: Haiku 4.5(경량) · Sonnet 5(이해/작업) · Opus 4.8(예외 에스컬레이션)
품질 체계	골든셋 64문항 회귀(라우팅 100%-recall 1.0) · 인용 기계검증 · 답변 감사자(judge) · 반례→평가문항 승격
프론트	Next.js 16 + React 19 + TypeScript + Tailwind v4 — 정적 export를 FastAPI가 동일 출처 서빙
백엔드	FastAPI + 자체 QueryEngine(상태 라우팅) · NDJSON 2단계 스트리밍(첫 응답 3.2초)
인프라	AWS EC2 t3.xlarge 단일 서버 + Docker 4컨테이너(App·OpenSearch·PostgreSQL·Caddy), 도메인+Let's Encrypt
자동화	EventBridge 자동 기동/종료 + 일일 증분 파이프라인(수집→파싱→임베딩→Fact) — 완전 무인
비용	2개월 약 \$115(예산 \$150) — GPU→CPU 전환·시간제 운영·Batch 50%·프롬프트 캐싱

▶ 쉬운 설명 — 이 표를 읽는 법

윗줄 셋(데이터·검색·품질)이 서비스의 본질입니다 — '무엇을 알고 있고, 어떻게 찾고, 왜 믿을 수 있나'. 아랫줄들(프론트~비용)은 그것을 세상에 내놓는 방법입니다. 면접에서 아키텍처를 물으면 이 순서로 답하는 것을 권합니다: 데이터 → 검색 → 품질 → 서빙 → 운영.

2. 기술 스택 전체 지도 (버전 명시)

2.1 언어·런타임

기술	버전	쓰임	선택 이유 + 경험
Python	3.11.15 (Conda env dart-rag)	파이프라인·백엔드 전부	데이터·ML 생태계 표준. 경험: Windows에서 lxml→torch 임포트 순서가 segfault를 일으킴을 발견, 'ST 먼저' 규칙을 코드 주석으로 고정
TypeScript	5.x	새 프론트엔드	API 계약(이벤트 8종·카드 필드)을 타입으로 고정 — 리팩터 안전망
Node.js	22.22 (개발기만)	Next 빌드 도구	서버에는 없음 — 정적 export 배포라 런타임 불필요
SQL (PostgreSQL 방언)	PG 16	Fact Store 전수 스크리닝	'전부 찾아줘'는 검색이 아니라 조회 — jsonb·윈도우 함수 활용

2.2 데이터·ML

기술	버전	쓰임	선택 이유 + 경험
BGE-M3 (BAAI)	st 5.4.1 로드, 1024차원	임베딩(dense)	한국어 강함+8192토큰+무료 자체호스팅. 경험: 모델 교체보다 청킹 개선(v2)이 효과적임을 실측 — 표 완결률 80→100%
BGE-reranker-v2-m3	CrossEncoder	상위 후보 재정렬	질문-문서 쌍 직접 채점으로 정밀도 ↑. 서빙은 CPU 로드(GPU 경합 회피)
OpenSearch	2.18.0 (+nori 플러그인)	BM25+kNN 하이브리드 색인	어휘·의미 검색을 한 엔진에서 + 메타 필터. 경험: nori는 이미지에 구워 넣어(Dockerfile) 재생성 사고 차단
PostgreSQL	16	Fact·재무·대화·피드백 8테이블	환각 0의 원천 — 정형 사실은 SQL이 답한다
PyTorch	2.6.0+cu124	임베딩 모델 실행	로컬 RTX 4060(8GB)로 60만 청크 7시간 — fp16으로 VRAM 절약

2.3 백엔드·인프라

기술	버전	쓰임	선택 이유 + 경험
FastAPI	0.136.3	API 서버 + 정적 서빙	타입 기반 검증(pydantic)-비동기 스트리밍. lifespan에서 엔진 1회 로드
Uvicorn	0.46.0	ASGI 서버	TLS 직접 서빙 가능 · NDJSON StreamingResponse
Docker Compose	29.4.0 엔진	4컨테이너 오케스트레이션	경험: Windows Docker 소켓 크래시루프를 표준 복구 절차로 다스림 · 서버는 재빌드 최소화 볼륨 설계
Caddy	2.x	리버스 프록시 + 정식 TLS	도메인 인증서(Let's Encrypt) 자동 발급·갱신 — 설정 몇 줄
Anthropic SDK	0.102.0	Claude 호출	tool use 구조화 출력 · Batch API · 프롬프트 캐싱 · max_retries

2.4 프론트엔드

기술	버전	쓰임	선택 이유 + 경험
Next.js	16.2.10 (Turbopack)	React 프레임워크	경험: dev rewrites 프록시가 80초 무데이터 스트림 꼬리를 유실 → Route Handler 프록시로 해결(대표적 트러블슈팅 스토리)
React	19.2.4	컴포넌트 UI	스트리밍 이벤트→상태 반영이 무료. 경험: 같은 틱 2회 setState는 stale — 테스트도 이를 고려
Tailwind CSS	v4	스타일	브랜드 토큰(@theme)으로 일관 팔레트 · 반응형 유틸리티로 1→3단 그리드
정적 export	BUILD_STATIC=1	배포 산출물	서버에 Node 없음 — FastAPI 동일 출처 서빙, 갱신=scp 30초

3. 서비스 아키텍처

사용자(브라우저/모바일)

| https (도메인: Caddy-Let's Encrypt / IP: 자기서명)

▼

 [EC2 t3.xlarge — Docker Compose 4컨테이너]

```
|— caddy      : 443 리버스 프록시 → app
|— app       : FastAPI(8000) = 정적 UI 서버(/=Next, /legacy=구UI)
|             + /api/* (질의-대화-피드백) + QueryEngine(6경로)
|             + 임베더/재랭커(CPU) + Claude API 호출
|— opensearch : 하이브리드 색인 604,458 청크 (nori 내장 이미지)
|— postgres   : Fact Store 13,523건·18종 + 대화·피드백·아카이브
```

▲ ▲

```
| EventBridge(평일 09시 기동/18:10 종료) cron(09:40 증분 · 18:00 종료)
|_____ AWS 제어면(EIP-SG-IAM-CloudWatch) _____
```

핵심 설계 판단: 마이크로서비스 대신 단일 서버 + 컨테이너 분리. 사용자 수십 명 규모의 평가용 서비스에서 분산의 복잡성은 비용일 뿐이다 — 대신 컨테이너 경계는 지켜 나중의 분리(관리형 OpenSearch/RDS 전환)를 열어뒀다.

▶ 쉬운 설명 — 아키텍처를 한 문장으로

한 대의 서버 안에 잘 구획된 네 개의 방입니다. 문(HTTPS)은 Caddy가 지키고, 응접실(app)이 손님을 맞아 서고(opensearch)와 장부실(postgres)을 오가며, 아침저녁 개폐는 AWS 알람시계가 합니다.

면접 포인트: '왜 이렇게 단순하게?'라는 질문에 — 규모에 맞는 복잡도 선택 + 분리 가능한 경계 유지, 두 가지로 답합니다.

4. 데이터 흐름도

4.1 적재 파이프라인 (매일 아침 무인 실행)

OpenDART(전시장 정기공시 조회, 최근 7일)

- 유니버스 808사 + 사업/분.반기 필터, [첨부정정] 회피
- 기색인 rcept 건너뛰(상태 저장소 = 색인 그 자체 → 먹등)
- 신규만: dcm해소(뷰어, 스로틀) → ZIP 다운로드(PK 매직넘버 검증)
- 파싱 v1(추출용) + v2(색인용, 표 자체완결 청킹)
- 임베딩(missing_only 이어받기) → OpenSearch
- (연차만) LLM Fact 12종 + 규칙 Fact 6종 → PostgreSQL
- 정정공시면: 옛 rcept 청크 삭제 + 옛 Fact는 facts archive 보관

4.2 질의 흐름 (6경로 라우팅)

질문 → [이해: Sonnet 5] 의도·필터 구조화 → [온톨로지+WICS 자동 인식] 정규화

- └ 전수 신호('전부/모두') → Fact Store SQL (환각 0, 2~3초)
- └ 수치·순위 → financials(XBRL) SQL 계산
- └ 관계('같은 감사인') → 지식그래프 순회(PG)
- └ 유사사례 → 필터 해제 순수 의미검색
- └ 개방형·스키마박 → 하이브리드 벡터(BM25+kNN+RRF+HyDE)
→ 재랭커 → [합성: Sonnet 5]
- 인용 기계검증(원문 대조) → [감사자: Haiku] 채점
- 반려 시 [Opus 재합성] → 재감사 → NDJSON 스트림으로 점진 전송

▶ 쉬운 설명 — 두 흐름의 관계

적재 파이프라인은 '밤사이 도서관 정리', 질의 흐름은 '낮의 사서 응대'입니다. 낮의 응대가 빠르고 정확한 이유는 밤의 정리(자체완결 청킹, 사실 선추출)가 좋기 때문입니다.

검색 품질 문제의 8할은 질의 시점이 아니라 적재 시점에서 생긴다는 것 — 이 서비스가 몸으로 배운 가장 큰 교훈입니다(v2 재임베딩 사례).

5. 데이터 모델 (ERD)

5.1 PostgreSQL — 관계 구조

runs (실행 이력)
└─< facts (사실 13,523 · 18종) —[정정 교체 시 복사]—> facts_archive
· corp_code / fiscal_year / fact_type / detail(jsonb)
· value_raw·unit_scale·currency·value_krw (단위 보존)
· evidence_text·section_path·rcept_no·dcm_no·dart_url (근거)
· confidence(ok|review = 인용 기계검증 결과)
financials (XBRL 정형수치: corp×연도×계정)
related_parties (특수관계자 그래프 엣지)
chat_threads ─< chat_messages (대화: state(jsonb)=기억, payload=답변 전체)
eval_candidates (반려→골든셋 후보, UNIQUE(question))
evidence_feedback (근거 채택/반려 로그)

테이블	핵심 컬럼	설계 의도
facts	fact_type + detail(jsonb) + 근거 5필드	유형별 스키마 차이를 jsonb로 흡수하되, 근거·단위는 공통 컬럼으로 강제 — '근거 없는 사실 없음'
facts_archive	facts 전체 + archived_at·superseded_by	정정공시로 교체된 옛 사실 보존 — '고친 것 자체가 정보'
chat_threads/messages	state(jsonb)·payload(jsonb)	프레임워크 없이 대화 맥락 영속화 — LangGraph 체크포인트의 자체 구현
eval_candidates	question UNIQUE·status	사용자 반려가 시험문제로 승격되는 파이프라인(하네스)

5.2 OpenSearch 인덱스(audit_chunks_v2) — 문서 스키마

필드	타입	역할
text	text(nori 형태소)	BM25 어휘 검색 대상 — 청크 본문(헤더+표 캡션 포함)
embedding	knn_vector 1024 (HNSW-cosine)	의미 검색 — _source 제외로 저장량 35% 절감
corp_code·fiscal_year·period_type·assurance·doc_type	keyword/integer	사전 필터(기업·연도·감사/검토·문서종류)
section_path·note_no·table_title	text/keyword	근거 위치 표기 + 섹션 가중 검색
rcept_no·dcm_no·dart_url·unit_hint	keyword	DART 딥링크·단위 추적 — 인용의 근간

▶ 쉬운 설명 — ERD 없이 jsonb를 쓴 이유

Fact 18종은 유형마다 담을 내용이 다릅니다(감사의견은 '의견', 소송은 '설명', 감사시간은 '숫자들'). 유형마다 테이블을 만들면 18개 테이블이 되고, 공통 컬럼만 두면 정보가 뭉개집니다. 절충이 '공통(근거·단위)은 컬럼, 유형별 내용은 jsonb'입니다. PostgreSQL의 jsonb는 인덱스·연산자가 풍부해 detail->>'auditor' 같은 구조화 질의도 SQL로 됩니다 — 유연성과 정합성의 균형점입니다.

6. 프론트엔드 (Next.js + React)

- 구조: 단일 페이지 + 탭 상태(검색/AI 대화) — lib/api.ts(NDJSON async generator)·types.ts(계약 타입)·components(SearchView·ChatView·EvidenceCard·FinTables·bits)
- 스트리밍 렌더: understanding→core(즉시 카드)→progress→supplement(이어붙임)→done 이벤트를 React 상태로 점진 반영 — 체감 응답 2~5초의 비밀
- 반응형: 전폭 레이아웃 + 카드 1→2→3단 그리드(md/2xl) + 본문 90ch 상한 — '공간은 정보 밀도로 환산'
- 배포: BUILD_STATIC=1 정적 export(865KB) → 서버 볼륨(scp 30초 갱신) → FastAPI 동일 출처 서빙, /legacy에 구 UI 보존(롤백 = 폴더 비우기)
- 겪은 것: ① dev rewrites 프록시의 장시간 스트림 꼬리 유실 → Route Handler로 해결 ② Windows 예약포트가 8000 점유 → 로컬 8200 우회 ③ 정적 빌드 전 dev 서버 중지 필수(파일 잠금) — 배포 게이트 신설

7. 백엔드 · 미들 계층

7.1 API 서버 (FastAPI)

- 엔드포인트: /api/query(비스트림)·/api/query/stream(NDJSON)·/api/chat/*(대화)·/api/meta(설정)·/api/login·/api/feedback·/api/evidence-feedback·/api/eval-candidates
- lifespan에서 QueryEngine 1회 로드(임베딩·재랭커·온톨로지·PG) — 요청마다 모델 로드하는 실수 방지
- 인증: X-Auth-Password 헤더(설정 시) · 접근/질의 감사로그 파일 · CORS 개방(프론트 분리 대비)
- 동시성: 모델 추론 직렬화 락(INFER_LOCK) = 단일 GPU/CPU 모델의 멀티유저 질의 큐 · PG 스레드로컬 연결

7.2 QueryEngine — 서비스의 심장

- 이해(understand, Sonnet 5 tool use) → 온톨로지 정규화(_apply_ontology: 업종 정확화·부정 집합·값 동의어·WICS 자동 인식) → 6경로 분기(run/run_stream)
- 하이브리드 검색: 멀티쿼리 4~6개 배치 임베딩(GPU락 1회) → BM25+kNN 각각 → RRF 융합 → HyDE 추가 검색 → 크로스인코더 재랭킹
- 이중 답변: 정리 데이터 코어(즉시) + 온디맨드 보완(이어서) — 커버 기업 제외·보일러플레이트 스킵·중복 제거
- 하네스: 인용 기계검증(_norm substring) → 감사자(judge) 채점 → 반려 시 Opus 재합성 → 재감사(rechecked)

7.3 미들(사이) 계층이 하는 일

구성요소	역할	비고
Caddy	443 TLS 종단·도메인 인증서 자동 갱신·역프록시	'미들웨어'라는 별도 서버 없이 프록시 한 층으로 충분한 규모
Uvicorn(ASGI)	비동기 요청 처리·스트리밍 응답	동기 프레임워크였다면 80초 스트림 동안 워커가 잠김
Next Route Handler (dev)	개발 중 API 프록시(스트림 통과)	운영에선 동일 출처라 프록시 자체가 없음 — 계층 최소화
Docker 네트워크	컨테이너 간 서비스명 통신(app→opensearch:9200)	호스트 포트 노출 최소화 = 공격면 축소

▶ 쉬운 설명 — 미들 계층의 철학 — 없는 것이 최선, 있어야 하면 얇게

요청이 거치는 층이 늘수록 장애 지점·지연·설정이 늘어납니다. 감사렌즈는 '브라우저 → Caddy → FastAPI' 세 층이 전부이고, 그마저 운영에선 화면과 API가 같은 출처라 CORS·프록시 문제가 원천적으로 없습니다.

개발 중 Next 프록시의 스트림 유실 사건이 좋은 반례 교재입니다 — 층 하나가 늘면 그 층의 버그도 함께 들어옵니다.

8. 청킹 전략 — v1의 실패와 v2의 설계

8.1 v1: 무엇이 문제였나 (818,507청크 전수조사로 발견)

문제	규모	결과
고정 길이 분할로 표가 중간에 잘림	연속 청크 79.7%	숫자 절단 19.6만 건 — 표 하반기가 '주인 없는 숫자'
잘린 표 조각에 칼럼 제목 없음	헤더행 유실 32.7만	'12,345 8,901'만 남아 무슨 수치인지 알 수 없음
각주가 표와 분리	13.2만 건	'(*) 상기 금액은...' 각주가 다른 청크로
행 칼럼 수 불일치	84.1%	병합셀(rowspan) 전개 없이 직렬화

8.2 v2: 구조 우선 청킹(structure-first)

- 경계 = 구조: 글자 수가 아니라 제목(TITLE)·표(TABLE) 단위로 1차 분할 — 설계서의 'section-aware'를 실데이터로 급진화
- 표 자체완결: 표는 통째로, 길면 행 단위로 나누되 조각마다 표 캡션 「[표: 제목](단위: ...)」 + 칼럼 제목행을 반복 주입 — 어느 조각을 검색해도 맥락 완비
- 병합셀 전개: rowspan 값을 각 행에 반복(_expand_table 재사용) — 칼럼 정합 복원
- 각주 결합: 표 뒤 (주N)※ 패턴은 표 청크에 붙임
- 패킹 버퍼: 짧은 표+주변 문장을 합쳐 청크 폭증 방지 — 결과: 청크 수 26% 감소(81.8만→60.6만)에 정보는 더 온전
- 컨텍스트 헤더: [회사·연도·감사/검토·연결/별도·섹션경로] 프리픽스 — 검색·인용 정확도의 기본기(설계서 계승)

검증: 무API 벤치(원문 대조로 정답 실존 확인한 22문항)에서 hit@5 동물(무회귀) + 표 자체완결률 80%→100%. 모델은 그대로인데 '먹이는 방식'만 바꿔 얻은 개선 — 임베딩 품질의 병목이 어디인지 보여준 실험이다.

▶ 쉬운 설명 — 청킹이 왜 검색 품질을 좌우하나

임베딩 모델은 받은 텍스트 조각만 보고 좌표를 찍습니다. 조각이 '주어 없는 숫자 더미'면 아무리 좋은 모델도 엉뚱한 좌표를 찍습니다.

요리에 비유하면 모델은 요리사, 청킹은 재료 손질입니다. 감사렌즈의 v2는 '요리사를 바꾸지 않고 손질을 바꿔' 품질을 올린 사례이고, 이는 실무 RAG의 정석 순서(데이터 먼저, 모델은 나중)이기도 합니다.

9. 임베딩·검색 전략

- 모델: BGE-M3(1024차원, 8192토큰) 자체 호스팅 — 선택 근거: 한국어 성능·긴 컨텍스트(표 완결 청크 수용)·대량 임베딩 호출비 0. 검토 후 기각: 삼일PwC 공개 모델(514토큰 한계로 v2 청크가 잘림)

- 하이브리드: BM25(nori 형태소) + kNN(HNSW cosine) → RRF 융합 — 회계 용어는 '정액법 vs 정률법'처럼 글자 하나가 의미를 가르므로 어휘 검색이 필수(설계서 판단이 실전에서 그대로 유효)
- 멀티쿼리(RAG-Fusion): 이해 단계가 동의어·하위개념으로 4~6개 질의 생성 → 각각 검색 후 융합
- HyDE: 개방형 질의는 Haiku가 '정답처럼 생긴 가상 문단'을 생성, 그 임베딩으로 추가 검색
- 재랭킹: BGE-reranker-v2-m3 크로스인코더 — 후보 k×3을 질문-문서 쌍으로 재채점
- 사전 필터: 기업·업종(corp_code 환원)·연도·감사/검토·문서종류를 검색 전 적용 — 정확도·속도·비용 동시 개선
- 증분 전략: missing_only(문서ID 존재 확인 후 빠진 것만) — 재실행 안전(먹등)의 근간

▶ 쉬운 설명 — RRF(Reciprocal Rank Fusion)

두 심사위원(어휘·의미)의 점수를 그대로 합치면 척도가 달라 왜곡됩니다. RRF는 점수 대신 '등수'만 봅니다 — 각 목록에서 $1/(60+\text{등수})$ 를 부여해 합산.

등수 기반이라 척도 문제가 없고, 여러 질의(멀티쿼리) 결과를 합칠 때도 같은 공식을 재사용합니다. 단순한데 강해서 실무 하이브리드 검색의 사실상 표준입니다.

10. LLM 오케스트레이션과 하네스

- 4계층 모델: Haiku 4.5(HyDE·보완 합성·감사자·엔티티 추출) / Sonnet 5-이해(라우팅·재작성) / Sonnet 5-작업(본답변·Fact 추출) / Opus 4.8(반려 시 재합성만) — '읽히는 글에는 매니저, 중간 산출물엔 사원'
- 구조화 출력: 모든 LLM 호출은 tool use(JSON 스키마) — 자유 텍스트 파싱 금지
- 인용 기계검증: 답변 인용문이 근거 원문에 실재하는지 정규화 substring 대조 — LLM 자기신고 불신(설계서 원칙)
- 답변 감사자(judge): 생성-평가 분리. 실전 사례: '약 1,800~1,900억대' 표현이 근거 수치와 미세히 어긋남을 감사자가 자발 반려 → Opus 재작성 → 재감사
- 골든셋 회귀: 64문항(정답을 원문 열람으로 직접 확정)이 모든 변경의 합격선 — 모델 교체(Sonnet 5)도 이 게이트 덕에 하루 만에 안전 완료
- 비용 레버: 프롬프트 캐싱(시스템·툴 정의 캐시 → 재사용 단가 1/10) · Batch API(대량 추출 50% 할인) · 이해 캐시(_uc_ver: 모델명 포함 → 교체 시 자동 무효화)

▶ 쉬운 설명 — 하네스(harness)라는 관점

에이전트의 품질 = 모델 × 하네스(모델을 둘러싼 검증·게이트·피드백 구조)입니다. 감사렌즈는 모델은 사서 쓰되 하네스는 전부 자체 설계했습니다 — 기계검증·감사자·골든셋·반려 승격.

감사인의 언어로는 '내부통제를 설계하고 운영 테스트까지 한 것'입니다. 면접에서 AI 품질을 물으면 모델 이름이 아니라 이 하네스를 답하세요.

11. AWS 인프라 요약

서비스	역할	한 줄 경험
EC2 (g4dn→t3.xlarge)	본체	타입 교체(가역)로 비용 60% 절감 — '측정 후 결정'
EBS 150GB gp3	영속 디스크	꺼도 데이터 생존 = 시간제 운영의 전제 (정지 중에도 과금)
Elastic IP	고정 주소	이력서 링크의 생명 — 재시작 IP 변동을 겪고 도입
Security Group	방화벽	가정용 IP가 하루 4회 변동 — SSH 전 SG 갱신이 절차화됨
IAM	권한	최소권한이 작동한 실화: 배포 계정으로 무인화 불가 → 콘솔 1회 승인 설계

서비스	역할	한 줄 경험
EventBridge Scheduler	무인 개폐	타임존 지원·월 44회는 무료 한도의 0.0003% · CloudShell엔 AWS_PAGER="" 필수
CloudWatch / Service Quotas / Secrets Manager	관측·한도·비밀	GPU 쿼터 0→승인 대기를 일정에 반영하는 것이 실무 감각

상세 학습은 별도 산출물 「AWS × AI Implementation 스터디」(14쪽 — 사용 9종 해부 + 확장 16종 + 하드웨어 기초)를 참조.

12. 설계 대비 변경 이력 — '계획과 현실' (면접 최고 소재)

항목	최초 설계(6/24)	실제 구현	바뀐 이유(경험)
LLM 라인업	Haiku 4.5 / Sonnet 4.6 / Opus 4.8	Haiku 4.5 / Sonnet 5 / Opus 4.8	신모델 출시 → 골든셋 게이트 덕에 1일 안전 교체. 라우팅 100% 달성
오케스트레이션	LangGraph + 멀티에이전트 + MCP	자체 QueryEngine(상태 라우팅) + 하네스	단일 프로세스 규모에서 프레임워크는 간접비용 — 필요한 개념(상태·검증 루프·체크포인트)만 직접 구현. 대화 저장소가 체크포인트 역할
검색 경로	3경로(벡터/Fact/온디맨드)	6경로(+그래프·수치·유사사례) + 방향 B(전수 신호 게이트)	실사용에서 관계·수치·유사 질의가 별도 최적 경로를 요구함을 확인
배포	S3-ECS Fargate-Step Functions-Aurora-API GW	EC2 단일 서버 + Docker + cron/EventBridge	예산 \$150/2개월 — 관리형 스택은 월 수백 달러. 규모에 맞는 복잡도로 하향, 분리 경계는 유지
파이프라인 트리거	사람이 수동 시작	완전 무인(자동 기동→증분→종료)	평가 기간 상시성 요구 → 설계보다 진화. 맥등·실패격리가 무인의 전제
임베딩	BGE-M3 (A/B 후보 3종)	BGE-M3 확정 + v2 재임베딩	모델 교체 대신 청킹 재설계가 효익 큼을 전수조사로 입증
산업분류	KRX 1차 + DART 교차검증	KRX + WICS 3단(공식 281·검증 추정 527) 이중 체계	'운송장비·부품' 문침 실사용 불만 → 공식 매핑 도입 + 원문 딥리서치로 125건 교정
Fact 12종	LLM 추출 12종	18종(LLM 12 + 규칙 6)	표준 서식 표는 규칙이 더 정확하고 \$0 — 감사시간·비감사용역·재작성 등 회계사 고유 지표 신설
프론트	S3+CloudFront (또는 Streamlit)	바닐라 HTML → Next.js+React 정적 export	1인 운영은 단순 우선으로 시작, 검증 후 현대화 — 병행 개발/legacy 롤백으로 무위험 전환

▶ 쉬운 설명 — 변경 이력이 면접에서 강한 이유

설계대로 다 됐다는 이야기는 신뢰를 얻지 못합니다. '이 가정이 실데이터/예산/사용자 앞에서 이렇게 깨졌고, 이렇게 조정했다'가 실무 역량의 증거입니다.

특히 LangGraph·관리형 스택을 '알고도 안 쓴' 결정은 유행 기술 수집가가 아니라 규모에 맞는 선택을 하는 엔지니어링 감각으로 읽힙니다 — 단, '필요해지는 시점'(멀티프로세스 오케스트레이션·수익화 규모)을 함께 말할 수 있어야 합니다.

13. 면접 Q&A; 10선

예상 질문	답변 골자(우리 실화 기반)
왜 RAG인가? 파인튜닝이 아니라?	근거 추적성(원문 인용+답링크)이 요구사항 — 생성이 아니라 '찾아서 보여주기'가 본질. 파인튜닝은 병목이 실증된 뒤의 수단(로드맵 보유)
환각은 어떻게 막나?	3중: ①전수 질의는 SQL(생성 자체가 없음) ②인용 기계검증(원문 substring 대조) ③독립 감사자 채점 — '수치 표현 부정확'을 자발 반려한 실사례 보유
'전부 찾아줘'를 벡터 검색으로 왜 못 하나?	top-k는 표본이지 전수가 아님 — 그래서 적재 시점 사실 추출(Fact Store) + SQL. 검색과 조회의 구분이 이 서비스의 핵심 설계
하이브리드 검색을 쓴 이유?	'정액법/정률법'처럼 한 글자가 의미를 가르는 도메인 — 의미 검색만으론 위험. BM25(nori)+kNN을 RRF로 융합
임베딩 품질을 어떻게 올렸나?	모델 교체가 아니라 청킹 재설계 — 82만 청크 전수조사로 표 절단을 정량화하고, 표 자체완결 청킹으로 완결률 80→100%(자체 벤치로 무회귀 확인)
LangGraph 같은 프레임워크는 왜 안 썼나?	단일 프로세스 규모에선 간접비용 — 필요한 개념(상태 라우팅·검증 루프·대화 영속화)만 직접 구현. 덕분에 개념을 바닥부터 설명 가능
비용 관리는?	예산 \$150/2개월 제약을 설계 입력으로: GPU→CPU 실측 전환·시간제 무인 운영·Batch 50%·캐싱·모델 티어링 — 최종 ~\$115
품질 회귀는 어떻게 막나?	골든셋 64문항(정답을 원문 열람으로 직접 확정)이 모든 변경의 배포 게이트 — 모델 교체·재임베딩·기능 추가 전부 이 관문 통과
가장 어려웠던 버그는?	후보 3개: ①Windows에서 임포트 순서 segfault(ST→torch 규칙화) ②dev 프록시의 80초 스트림 꼬리 유실(Route Handler로 해결) ③모델 교체 후 recall 붕괴가 실은 낮은 채점 기준(가짜 회귀) — 셋 다 원인 분리 과정을 말할 수 있음
회계사인데 왜 이걸 만들었나?	감사 실무의 병목(전수 스크리닝·근거 추적)을 아는 사람이 도구를 설계할 때 가장 정확함 — 감사시간·비감사보수·재작성 같은 지표 선정 자체가 도메인 지식의 산물

14. 기술 용어 사전 (이 서비스를 설명하는 데 필요한 말들)

용어	쉬운 정의 + 우리 서비스에서의 위치
RAG	검색으로 근거를 모아 그 안에서만 답하게 하는 구조 — 서비스 전체의 뼈대
임베딩/벡터	글→숫자 좌표(1024개). 비슷한 뜻=가까운 좌표 — BGE-M3가 담당
청킹	문서를 검색 단위로 자르는 일 — v2의 '구조 우선·표 자체완결'이 우리 차별점
BM25 / kNN / 하이브리드	어휘 일치 점수 / 좌표 근접 검색 / 둘의 결합 — RRF로 융합
재랭커(CrossEncoder)	질문-문서 쌍을 직접 읽고 재채점하는 2차 심사위원
HyDE	가상의 모범답안을 만들어 그 좌표로 검색 — 개방형 질의 보조
tool use(구조화 출력)	LLM 답을 JSON 스키마로 강제 — 자유 텍스트 파싱 제거
NDJSON 스트리밍	한 줄=한 JSON 이벤트로 흘러보내기 — 점진 렌더의 전송 형식
멱등(idempotent)	몇 번 실행해도 결과 동일 — 무인 파이프라인의 제1조건
프롬프트 캐싱 / Batch API	반복 프리픽스 재사용(1/10 단가) / 비실시간 일괄 50% 할인
골든셋 / 회귀 테스트	정답지 문항 / 변경 후 다시 채점해 퇴행 감지 — 우리의 배포 게이트
하네스	모델을 둘러싼 검증·게이트·피드백 구조 — '모델 × 하네스 = 에이전트'
jsonb	PostgreSQL의 구조화 JSON 컬럼 — Fact 18종의 유형별 차이를 흡수
정적 export / 동일 출처	서버 없이 열리는 빌드 산출물 / 화면·API 같은 주소 — 배포 단순화의 열쇠

용어	쉬운 정의 + 우리 서비스에서의 위치
RRF	점수 대신 등수로 결과를 합치는 융합 공식 $1/(60+\text{rank})$
LoRA	본체 동결+작은 노브 학습 — 파인튜닝 로드맵의 수단

15. 사용 라이브러리 전체 목록

Python (dart-rag 환경, Python 3.11.15)

라이브러리	버전	용도
fastapi / uvicorn	0.136.3 / 0.46.0	API 서버 / ASGI(스트리밍·TLS)
anthropic	0.102.0	Claude API(tool use·Batch·캐싱·재시도)
sentence-transformers / transformers / torch	5.4.1 / 4.55.4 / 2.6.0+cu124	임베딩·재랭커 로드/추론
opensearch-py	3.2.0	색인·하이브리드 검색·delete_by_query
psycopg[binary]	3.3.4	PostgreSQL(스레드로컬 연결)
lxml	6.0.4	DART XML 파싱(recover 모드) — 파서 v1/v2의 기반
pydantic	2.13.4	요청 검증·설정(BaseSettings)
requests / httpx	-	OpenDART·KRX 호출(레이트리팅·재시도 래퍼)
boto3	-	Secrets Manager(준비)·AWS 연동
pypdf / reportlab	-	산출물 PDF 병합/생성(문서 자동화)
PyYAML / numpy	- / 1.26.4	온톨로지 로드 / 수치 처리

JavaScript (web-next)

라이브러리	버전	용도
next / react / react-dom	16.2.10 / 19.2.4	프레임워크·UI
typescript	5.x	타입 안정성(API 계약 고정)
tailwindcss(@tailwindcss/postcss)	v4	유틸리티 CSS·브랜드 토큰
eslint	9.x	정적 검사(빌드 게이트의 일부)

의존성 원칙: 상태관리·UI킷·HTTP클라이언트 등 추가 라이브러리 0 — 표준 fetch·React 혹은 충분한 규모에서는 의존성 하나가 곧 유지보수 부채라는 판단. requirements.txt는 로컬 검증 버전으로 고정(pin)해 서버와 완전 일치.

맺음 — 이 문서의 모든 항목에는 '왜'와 '겪은 것'이 붙어 있다. 기술 목록은 검색하면 나오지만, 그 기술이 실데이터와 예산과 사용자 앞에서 어떻게 조정됐는지는 우리만 말할 수 있다. 스터디는 12장(변경 이력)과 13장(Q&A;)을 먼저, 세부는 해당 장으로 돌아와 채우는 순서를 권한다.

16. 에이전트 관점 정리 — "에이전트 쓰셨나요?"에 답하는 법

Anthropic 공식 분류(Building Effective Agents)는 워크플로(코드가 정한 경로를 LLM이 따라감)와 에이전트(LLM이 다음 행동을 스스로 결정)를 구분한다. 감사렌즈는 이 스펙트럼의 중간 — 완전 자율 루프는 없지만, LLM이 '판단'하는 지점 4곳에 공식 에이전틱 패턴이 들어가 있다.

패턴(공식 명칭)	우리 구현	위치
라우팅(Routing)	이해 단계 LLM이 질문 의도를 분석해 6경로 중 행선지 결정 — '다음 행동을 모델이 고르는' 핵심 요소의 1회성 수행	claude.py understand → query.py 경로 분기
평가자-최적화 루프(Evaluator-Optimizer)	생성-평가 분리: 감사자(judge) 채점 → 반려 시 Opus 재합성 → 재감사. 가장 에이전트다운 부분	judge_answer → 에스컬레이션 재채점
쿼리 플래닝	LLM이 검색 계획 수립 — 동의어·하위개념 멀티쿼리 4~6개 + HyDE 가상 문단	query.py 멀티쿼리·HyDE
대화 메모리	스레드 state(jsonb)에 기업·계정 맥락 영속화 → 후속 질문 재사용 — LangGraph 체크포인트의 자체 구현	chat_threads.state + rewrite_followup

16-1. 왜 완전 자율로 가지 않았나 (핵심 답변)

- 감사 도메인은 예측 가능성·근거 추적이 최우선 — 자율 루프는 비결정적이고 비용·지연이 통제되지 않는다.
- 그래서 '판단은 모델에, 궤도는 코드에' — 모델의 판단마다 검증 장치(인용 기계검증·감사자·골든셋)를 부착.
- 공식: 에이전트 = 모델 × 하네스. 모델은 사서 쓰되 하네스(검증·게이트·피드백 구조)는 전부 자체 설계 — 감사인의 언어로는 '내부통제를 설계하고 운영 테스트까지 한 것'.
- 예상 꼬리질문 — "그럼 언제 자율 에이전트가 필요한가?": 도구 선택지가 동적으로 늘어나거나(예: 임의 외부 시스템 조회), 단계 수를 미리 알 수 없는 탐색 문제일 때. 현재 6경로는 닫힌 집합이라 워크플로가 정답.

▶ 쉬운 설명 — 자율 에이전트 vs 우리 방식

자율 에이전트는 '목적지만 주면 알아서 운전하는 기사', 우리는 '갈림길마다 기사에게 물어보되 지도는 회사가 줬 방식'입니다. 갈림길 판단(라우팅), 백미러 확인(감사자 루프), 경로 탐색(쿼리 플래닝), 지난 손님 기억(대화 메모리)은 기사가 하지만 운전대를 통째로 넘기지 않았습니다.

면접 전략: '에이전트 안 썼습니다'가 아니라 '에이전틱 패턴 4개를 워크플로 안에 통제된 형태로 썼고, 자율 루프를 안 쓴 이유를 설명드리겠습니다'로.

17. 사례연구 — 라이브 결함 2건의 전 과정 (2026-07 실화)

면접에서 '가장 어려웠던 버그'류 질문에 쓸 수 있는 최신·최상 소재. 증상→진단→수리→재발방지의 완결 루프를 전부 실측 근거로 말할 수 있다.

17-1. 증상과 진단 — 추측 대신 실측

- 증상: 사용자 화면에 ①답변 속 <answer>·</analysis> 태그 노출 ②우발채무 표가 '빈 셀 바다에 숫자 몇 개'로 렌더.
- 진단 1(태그): 앱 로그에서 해당 질의 특정 — 도구 호출 성공·인용검증 8/8·에스컬레이션 없음. 즉 파싱 실패가 아니라 모델이 JSON 문자열 필드 안에 응답 전체를 태그 형식으로 재서술한 드리프트(장문·표 위주 근거 + 번호 템플릿식 프롬프트가 유발). 3중 방어가 전부 '내용'만 검사하고 '형식'은 안 봄 — 하네스의 사각지대.

- 진단 2(표): PG 격자 덤프 — 주식 하나에 하위 표 ~20개, 그중 약정 유형 27개가 열로 전개된 축 전도 격자·무헤더 레코드 실증. 표를 '잘못 골라온' 게 아니라(선택 로직 정상) '구겨진 채 저장'돼 있었다.

17-2. 수리 — 결함의 성격에 맞는 도구 선택

결함	선택한 수리	기각한 대안과 이유
태그 유출	정규식 후처리 가드 _sanitize_synt(무API-결정적·공짜) + 프롬프트 방어 1줄 + 챗 저장분은 조회 시점 정화	LLM 재호출 루프(LangGraph류) — 비용·지연·비결정성. 코드로 잡을 결함에 LLM을 부르지 않는다
표 격자 붕괴	3중: ①렌더 게이트(_grid_ok — 빈셀 60%·무헤더 차단→LLM 정규화 폴백) ②추출 수리(중복열 그룹 구별·무헤더 제외·전치 복원) ③전수 재추출·교체	게이트만으로 끝내기 — 노출은 막지만 원본 품질은 그대로. 사용자가 재구축 승인 → 근본 수리

17-3. 재구축 — 분석 먼저, 실행은 나중

- 전수 분석(96분·3,209필링·실패 0): 표 1,599,577개 지표화 — 게이트 탈락 1.9%(재무제표 본문 0.3%), 잔여 탈락은 계열출자·특수관계자 매트릭스형(격자 표시 부적합 → 폴백이 정답으로 '판정')
- 적재: COPY 80초 → 인덱스 → 트랜잭션 이름 스왑(무중단) → 검증 3중 통과 → 구 테이블 삭제(DB 4→2GB)
- 무결성 3중 대조: 추출 원장 = 적재 로그 = DB count 전부 1,599,577 일치 + 기업 808/808 커버리지
- 검증: 문제의 질의 재실행 — 깨졌던 표가 '공시 원본 표' 격자로 정상 서빙, 태그 0, 인용 9/9

▶ 쉬운 설명 — 이 사례가 면접에서 강한 이유

①증상을 화면이 아니라 로그·DB로 특정했고 ②결함의 성격(형식/데이터)에 따라 도구를 다르게 골랐으며 ③고치기 전에 전체를 재서(전수 분석) '남은 1.9%는 고칠 게 아니라 폴백이 정답'이라는 판단까지 데이터로 했습니다.

'버그를 고쳤다'가 아니라 '검증 체계의 구멍을 찾아 메웠다'로 이야기가 끝나는 것 — 이게 주니어와 시니어 답변의 차이입니다.

18. 면접 Q&A; 추가 5선 (13장에 이어서)

예상 질문	답변 골자(실화 기반)
LLM이 형식(스키마)을 어길 때 어떻게 대응하나?	tool use로 스키마를 강제해도 드물게 문자열 필드 안에 태그 재서술 드리프트 발생(실측). 대응 3중: ①프롬프트 방어 ②결정적 후처리 가드(정규식 회수 — 공짜·즉시) ③영속 데이터는 조회 시점 정화. 재호출 루프는 마지막 수단
LLM이 만든 표를 어떻게 신뢰하나?	신뢰하지 않는다 — 원본 XML 격자(스토어)가 1순위, LLM 정규화는 폴백이며 그 결과도 모든 숫자를 원문 substring 대조(기계검증)로 확인, 실패 시 원문 발췌로 강등. '생성물은 검증 후에만 승격'
대량 데이터 교체를 무중단으로 하려면?	신테이블 적재 → 인덱스 → 트랜잭션 이름 스왑(원자적) → 검증 → 구테이블 삭제. 무결성은 원장=적재로그=DB 3중 대조 + 커버리지·분포 정합. 실사례: 160만 행 교체, 서비스 중단 0
자동화된 인프라에서 수동 작업하다 겪은 문제는?	야간 수동 적재가 자동 종로 스케줄과 2회 충돌해 작업이 끊김 → 컨테이너 분리 실행(docker exec -d + 완료 마커 + 폴링)으로 세션 독립성 확보. 교훈: 무인 자동화가 좋아질수록 '사람이 개입하는 날의 절차'가 별도로 필요
데이터 품질 이슈를 어디까지 고치나(수리 vs 수용)?	전수 분석으로 분포를 먼저 켜다 — 잔여 결함 1.9%가 '매트릭스형 표'라는 성격임을 확인하고 수리 대신 폴백 경로로 '수용' 판정. 모든 결함을 고치는 게 아니라 결함의 성격별로 처리 경로를 설계하는 것

스터디 순서(갱신): 12장(설계 대비 변경) → 13장(Q&A;) → 17장(사례연구) → 16장(에이전트 관점) → 18장(추가 Q&A;) 순을 권한다. 사례연구는 다른 모든 장의 개념(하네스·게이트·폴백·전수조사)이 실전에서 한 번에 작동한 종합 문제풀이다.